对。现在应该先形成一份**基于实际实验的能力与缺口清单**，再决定 CA 保留、删减或加强什么。

我重新核对了最近 HashiCorp、Dragonboat、OmniPaxos 的对照实验，并补查了 EPaxos、ZooKeeper、OpenRaft 的 DeepSeek baseline。结论是：

baseline 已经具备一条相当完整的研究能力链；主要问题是运用不稳定。CA 在部分路径上提高了研究深度和纠错质量，但没有全面解决这些不稳定性，还增加了能力限制、记录维护和局部研究过度集中的成本。

下面把这些区别展开。

baseline 已经能够独立完成的事情，比我们最初假定的多。

能力	已有实际表现	应如何认识
发现跨路径的共识问题	HashiCorp 从启动重放与配置扫描的交互发现成员恢复错误；OmniPaxos 从编码缓存状态转移发现副本决定值不同	不是只能发现 API 边界错误
构造实际协议历史	HashiCorp 使用正常快照、成员添加、写入、关闭和重启；OmniPaxos 使用真实实例和生产消息	可达性研究不是 CA 独有
编写复杂实验	多节点消息调度、分区、恢复、随机测试、持久存储重开、实际客户端调用	已经具备较强的自主实验能力
使用对照判断原因	Dragonboat 对比读请求入口、唤醒行为，并在独立诊断副本中验证候选修正	不只是“测试失败，因此存在缺陷”
使用反证排除怀疑	EPaxos 排除正常生产路径无法产生的冲突状态；OpenRaft 找到上游保护，撤回成员回退怀疑	反证和修订能力已经存在
保留范围与负面结果	多份报告区分局部接口问题、源码推断、实际执行及未证实的集群后果	baseline 也能够谨慎解释

几个最能说明问题的例子：

- **HashiCorp baseline 的成员恢复问题**，最终通过正常集群操作，观察到两个分区在相同索引、相同任期提交不同数据。这已经是较强的实际协议见证。原始结果
- **OmniPaxos baseline 的 LFU 问题**，真实三个实例在重新同步后，对同一位置分别决定了 `A` 和 `C`；另外还做了 LRU 对照。原始输出
- **Dragonboat baseline 的 quiesce 问题**，启动真实三个 NodeHost，等待静默、停止 Leader，再发客户端提案；结合源码、唤醒对照和独立诊断修正，追到了进展停滞的原因。分析与复现
- **OpenRaft baseline 的反证处理**，起初怀疑旧快照导致成员状态回退，随后发现真实入口的 committed guard，明确撤回怀疑，并按照合法条件生成检查。排除过程

因此，“模型不会发现复杂问题、不会验证合法前史、不会使用反证”已经不适合作为当前 CA 的设计前提。

但 baseline 的问题也很明确，主要集中在能力没有稳定贯彻到最终结果。

第一，**同一份报告里的证据标准不一致。**
它可以为一个问题建立真实多节点历史，也会把另一个手工构造异常消息的局部失败列成高等级问题。

例如 ZooKeeper 的一些检查使用 mock follower、手工消息和预设内部状态，确实触发了异常，但没有建立合法 CFT 生产者如何产生这些条件。Dragonboat baseline 的若干条目也属于畸形网络输入问题。它们不能与真实 CFT 历史下的协议错误直接合并计数。

第二，**局部纠错不一定同步到最终结论。**
EPaxos baseline 明确说某个 subcase 疑问不计为 confirmed defect，但摘要和前面的 Finding 部分仍保留了对应条目。它已经理解了反证，最终报告却没有完全收敛。报告中的最终限制

第三，**会把合理期待加强成没有充分依据的责任。**
例如“操作返回错误”是否意味着所有修改必须回滚，需要具体接口合同支持；有限等待没有结束，也不能单独支持永久停滞。baseline 有时区分得很好，有时直接跨过去。

第四，**持续调查能力波动很大。**
HashiCorp baseline 最后约 12 分钟连续重复总结，最终在约 77.7 分钟耗尽 200 回合；OmniPaxos 最后约 18 分钟也没有新的工具工作。但 EPaxos、OpenRaft 的末尾重复只有约一两分钟，不能把持续性不足说成每次都严重发生。

这些问题说明：**baseline 缺的不是某个研究步骤，而是正确研究行为的稳定性、结论的一致性和资源分配能力。**

CA 已经产生的实际增益，可以明确列出，但其中多数仍属于“有所改善”。

方面	CA 中发生的实质增益	尚未达到的程度
将局部现象推进到实际历史	OmniPaxos 决定索引回退，从条件构造推进到真实五节点、实际消息产生的 5→4→5	不能据此认为所有 CA 见证都具有合法前史
追到真实消费者后果	Dragonboat 从 ReadIndex 上下文错配推进到真实 pending-read 消费者、超时端点和周期心跳触发	其他主张仍可能只有源码推断
根据新证据修正解释	HashiCorp 发现重启实验漏接 transport，撤回“无法恢复”的解释，修好构造后观察到恢复	尚未稳定处理所有测量错误和解释扩张
保存修订与证据归属	原始输入、输出、当前解释和旧解释可以对应核查	保存正确不等于解释正确
延长有效调查	HashiCorp、OmniPaxos 在 baseline 已经重复总结的阶段仍有新实验和修正	Dragonboat 展现了相反情况

其中，Dragonboat 的增益尤其清楚：CA 找到了 baseline 没有发现的 **ReadIndex 确认合并时串错请求身份**。它没有停在“回复字段错了”，而是验证了真实消费者不能完成原请求，并补上正常周期心跳触发及对照。当前解释

HashiCorp 的纠错也是真实发生的：首次重启检查因为测试 transport 没连接而失败，修正后 `restarted_as_leader` 和 `peer_caught_up` 都变成 true，当前判断撤回了那部分恢复失败后果。修正后的观察

不过，baseline 自己也修正过错误测试。因此，CA 应争取的是**让这种纠错更稳定、更及时、更省成本**，而不是把纠错能力本身算作从零增加的能力。

CA 引入的问题，至少有四类。

1. **人为缩小了可用研究能力。**

   Cargo features 是已经确认的实例：baseline 可以研究某些构建变体，CA 当时不能正式选择它们，直接影响了可调查范围。当前代码已经修正，但这项修正还没有经过新的自主实验验证。

   还有接口可发现性问题：OmniPaxos CA 认为 runtime crate 无法执行，实际上已有切换 crate 的能力。结果它用复制的游标计算代替真实 actor，而 baseline 直接运行了 actor。这里 CA 的证据反而更弱。

2. **产生了 baseline 不需要承担的维护工作。**

   当前 CA 要维护问题、知识、解释和引用关系；修改知识与解释时需要重交完整当前内容，反馈中又重复呈现已有记录。

   Dragonboat 后期有多次更新主要在“对齐问题引用与解释引用”。这些更新是实际工作，但没有改变目标实现的任何判断。不能把它们算成研究进展。一个完整的无变化提交

3. **容易让已有问题持续占据注意力。**

   OmniPaxos 的 43 次正式执行中，37 次位于同步／重配置相关方向；HashiCorp 的五个方向在前约 35 分钟已经提出，后面主要深化这些方向。

   其中有价值的深入不能删掉，但这也说明：当前“持续调查”经常表现为继续加工已有问题，未必转化成新的独立发现。重复呈现已登记内容可能强化这种倾向；这是需要验证的机制解释，不能仅凭执行分布断定全部原因。

4. **结构化知识也会保留错误，并成为后续研究依据。**

   OmniPaxos CA 把 `is_reconfigured()==None` 解释成 StopSign 不存在，但该 API 判断的是重配置是否已决定，不等于底层标记是否存在。错误解释进入了知识条目，并被问题引用。

   因而当前知识机制既保存了正确认识，也可能保存并继续使用误解。**“写成 Fact”没有使内容变成事实。**

尤其需要修正一个认识：持续性目前不能列为 CA 已经解决的问题。

对照目标	baseline 后期表现	CA 后期表现
HashiCorp	连续重复总结，提前耗尽调用预算	有更多后续执行和修正，但大量时间集中在原有五题
OmniPaxos	最后约 18 分钟无新工具工作	继续深入 StopSign，末尾重复较短
Dragonboat	最后约 96 秒才连续无工具总结	27 次无变化提交，模型耗时合计约 **12 分 35 秒**

Dragonboat 的 CA 在最后连续约八分半都没有语义变化。提交理由已经变成“问题都解释了、引用一致、记录完整”。这暴露了一个具体风险：**研究材料的完整性，正在成为模型判断“没有工作可做”的替代标准。**

所以，不能通过更严格地要求“继续调查”解决它。系统已经要求了，模型仍然可能继续提交空结果。

目前仍未解决的核心问题，可以收敛为下面五项。

未解决问题	主要归因	CA 当前能做什么、缺什么
合法历史与任意输入没有稳定区分	模型语义判断为主	能要求说明并保留前史，但不能确保它真的成立
观测字段或测试 oracle 被误解	模型与实验构造共同影响	能保存原始依据，尚不能稳定促使模型核对测量含义
局部现象被扩大成更强后果	模型判断为主	有范围、反证和解释机制，但仍会越过未验证的中间环节
纠错后相关结论没有同步收敛	模型判断与状态组织共同影响	修订可追踪；跨问题依赖纠错的实际收益还很有限
同根因反复研究、无法有效重新选题	模型选择与反馈设计共同影响	提供了全局记录导航，但尚未稳定带来独立研究增长

这次核对后，我对 CA 的判断更具体了：

它已经证明能够帮助模型把某些问题研究得更深，也能产生 baseline 漏掉的重要发现；但尚未证明现有整套组织方式稳定优于自由调查。 同时，执行能力不对齐、重复反馈和引用维护是明确的系统成本，应优先处理；合法前史、观测含义和责任判断则是模型仍不稳定的能力，应通过更有效的证据使用来改善，而不能无限增加字段和流程。

接下来需要保住的是这些已经发生过的有效深化和纠错，同时减少它们之外的维护成本。判断每项 CA 机制是否必要，应看它有没有改变研究选择、纠正结论或产生新发现，而不是看它能否让记录更完整。